

ATLAS — Practice Q&A

Likely supervisor questions with suggested answers, based on the ATLAS_v2 deck and checked against the repository and the final report (Final_report/chapters/evaluation.tex, design.tex). Answers are written to be said out loud: short first, detail only if asked.

READ FIRST

Fix these slide claims before presenting

These statements in ATLAS_v2.pdf do not match the repository. A supervisor catching an inflated number costs more credibility than any hard question, so correct the slide or be ready with the accurate version.

Slide claim	What to say instead
Slides 8–9: “10128 unique questions generated”	10,128 is the number of specifications (1,688 skills × 6 Bloom levels), not questions. Say: “about 12,300 verified questions, covering 94% (9,535) of the 10,128 skill × Bloom specifications.” Nothing shows they are “unique”.
Slides 8–9: diagnostic “stops at <15% of skills untested”	That rule exists in diagnostic.py but the server switches it off. The live diagnostic asks one question per skill in the section, capped at 30, at least 5 .
Slides 8–9: “Export research data — anonymised”	The export includes user_name in plain text. Either drop “anonymised” or say “contains no personal data beyond a username”. It is a current-mastery snapshot, not an answer log.
Slides 8–9: “Watch class progress — mastery overview per student”	The stats page shows class-wide aggregates (averages, histogram, weakest/strongest skills). Per-student data only comes through the export.
Slides 8–9: “Gate the next skill — opens only when parents ≥ 95%”	Nothing is locked from the student. The gate changes the learning rate : $T = 0.10$ when all parents are ≥ 0.95 , otherwise 0.01. Say “learning credit is gated”.
Slides 8–9: “loop until the parent is mastered”	Spillover injects exactly 2 prerequisite questions, then returns to the topic. Slide 24 already says this, so slides 8 and 24 contradict each other.
Slides 8–9: “practise the weak skill — routed to the real gap first”	Practice starts at the topic skill with the highest mastery still below 95% (the zone of proximal development). Routing to the gap happens only when spillover fires.
Slide 18: second prompt box	It repeats the generation prompt. The verifier prompt actually starts: “You are a strict subject-matter expert grader AND relevance judge for Bangladesh university admission exams... solve the question yourself from scratch using only the stem and options given.”
Slides 17–18: “Generate: Gemini 3.5 Flash-Lite”	True for the production run, but early pilot batches were generated with 3.1 Flash-Lite, and the generator doesn’t record the model per question. Safe wording: “production run on 3.5 Flash-Lite; early pilot batches on 3.1 Flash-Lite”. Verification is always 3.1.
Slide 5: “2 standard deviations (98%) better”	Bloom (1984): the average tutored student outperformed about 98% of students in a conventional class (about +2 SD). It is not “98% better”.
Slides 8–9: “Adaptive, in English or Bangla”	The interface switches EN/BN. Question content is Bangla only.
Slide 15 / slides 8–9: 1,743 edges	Correct as the raw count. What the engine traverses is 1,683 after transitive reduction. Worth saying both.
Live demo of the admin editor or validator	Three test skills are committed in Ontology/full_corpus_rebuild/tuples.json: notun_skill , notun_skill2 , skill (“sdf”) . The validator now reports 1,691 skills, 1 isolated skill. Remove them before demoing. The live catalog students use is clean.

SECTION 1

Motivation and the problem

Q1. What problem are you solving, in one sentence?

Students fail at a topic because of a specific earlier gap they never closed — the “Swiss cheese” problem — and classrooms at a 1:51 ratio can’t find that gap for each student. ATLAS finds the exact missing prerequisite and makes the student close it before moving on.

Q2. Who is the target user?

HSC-level students preparing for BUET, KUET, RUET and CUET engineering admission tests, in Physics, Chemistry and Mathematics. That is why the ontology is built from past admission-test questions and the content is in Bangla.

Q3. What is the 2 Sigma problem and why is it relevant?

Benjamin Bloom (1984) found that the average student with a one-to-one tutor outperformed about 98% of students taught in a conventional class — roughly two standard deviations. One-to-one tutoring doesn’t scale; ATLAS tries to automate the part of it that matters most: diagnosing what a specific student is missing.

If they push: don’t say “98% better” — it’s a percentile, not a percentage improvement.

Q4. How is this different from existing platforms like 10 Minute School or Shikho?

Most existing platforms deliver content: videos, notes, mock tests with a score. They tell a student they got a question wrong, not *why*. ATLAS models each skill separately, links skills by prerequisites, tags each wrong option with the prerequisite it points to, and redirects practice to that prerequisite. The unit is the skill, not the chapter.

SECTION 2

Building the ontology (the skill graph)

Q5. What is the ontology?

A directed acyclic graph where each node is one atomic, testable skill (for example “calculate the pH of a strong acid”), and each edge means “this skill is a prerequisite of that one”. It has **1,688 skills** and **1,743 prerequisite edges** (1,683 after removing redundant edges), covering all **183** syllabus topics, with a maximum depth of 9.

Q6. Where did the skills come from?

From **4,474 past admission-test questions** with worked solutions, across six question banks (two each for Chemistry, Mathematics and Physics). About 2,526 are MCQs and 1,948 are written questions, tagged from BUET, KUET, RUET, CUET and SUST papers. For each question the model asked: which atomic skills does solving this require?

Q7. Where did the 183 topics come from?

From the NCTB HSC syllabus: 51 chapters and 183 topics (Physics 72, Chemistry 59, Mathematics 52). Topics were placed where the NCTB book teaches them — for example gas laws sit under Chemistry 2nd Paper Chapter 1. We fixed the topics first so the extraction had a closed vocabulary to file skills into, instead of inventing topics as it went.

If they push: the old 430-skill ontology invented its own topics, and its topic density was a sampling artefact: Mathematics had 9.4 topics per 100 skills, Chemistry 23.1.

Q8. How did you go from 4,474 questions to 1,688 skills?

In four steps. **Extraction** in 27 batches read the questions and proposed skills and prerequisites. **Consolidation** cleaned it up: retagging, removing duplicates, splitting skills that bundled two things, authoring missing foundation skills, and adding edges. **Validation** checked it is acyclic and has no broken references. Most questions test skills that already exist, so the skill count is much lower than the question count.

Q9. What are the 27 batches?

The questions were processed in chunks of roughly 150–320, one subject at a time: Chemistry batches 1–9, Mathematics 10–18, Physics 19–27. After each batch the new skills were merged into the running ontology and validated before the next batch started, so an error was caught in the batch that caused it.

Q10. How did you avoid creating the same skill twice across batches?

The rule we called “reuse before minting”: before creating a skill, the model had to check the skills already extracted and reuse an existing ID if one covered it. Each batch was given the list of existing skills. After each batch a word-overlap script flagged near-duplicates for review; it never merged anything automatically. You can see it working: new skills per batch fell as a subject filled up — Chemistry went 189, 99, 32 across its last three batches.

Q11. What is “consolidation” exactly?

The editorial pass after extraction. In order: retagged 1,051 skills into the right topics; authored 40 foundation skills no question tested directly (like “solve a linear equation”); split 20 skills that fused two Bloom levels; merged 8 duplicates and added 10 cross-subject links; fixed validator findings; and added 48 edges linking small disconnected fragments. During extraction we also backfilled 352 Chemistry edges when that graph came out too sparse.

Q12. What did consolidation actually improve? Give numbers.

Empty topics went from 54 to 0, unresolved prerequisites from 46 to 0, isolated skills from 6 to 0, disconnected fragments from 91 to 32, and the largest connected component grew from 203 to 482 skills.

Q13. What does “validation” check?

That the graph has no cycles — a cycle would mean skill A needs B and B needs A, which breaks gating — and that every prerequisite points to a skill that exists, every skill has a topic, and cached descriptions match. The validator reports zero errors. Its warnings are review lists, not failures.

Q14. Why a DAG and not just a list of topics?

Because the whole point is to find the *earlier* gap. With only topics, if a student fails titration we can only say “titration is weak”. With a prerequisite graph we can say the real problem is mole calculation two levels below, and send them there.

Q15. Some skills are duplicated across subjects. Why not merge them?

HSC teaches some things twice — Higher Mathematics dynamics nearly repeats Physics kinematics. Merging would empty one of the two syllabus topics. We kept both and joined them with an edge from the subject that teaches it first, so mastery credit flows across through ancestor pull-up. That is why the validator reports 4 duplicate descriptions: they are deliberate.

Q16. Your checks flagged hundreds of problems. How did you handle that?

We read every flag instead of auto-fixing. The “Bloom inversion” check flagged 147 edges where the prerequisite was at a higher Bloom level than the skill; only 3 were really wrong, because Bloom level measures kind of thinking, not teaching order — you learn to evaluate a determinant before learning the fact that two equal rows make it zero. The multi-action check flagged 173 skills; 47 were real. Both heuristics over-flagged by a large factor.

If they push: the rule behind it: a wrong prerequisite edge silently corrupts mastery through pull-up, so a missing edge is safer than a wrong one.

Q17. How do you know the prerequisite edges are correct?

Honestly, structurally yes, pedagogically not fully. Every edge was proposed by the model with the question and solution in view, and we manually reviewed the flagged cases. But no chemist or physicist has reviewed the graph yet. We list that openly: 352 hand-authored Chemistry edges, 51 authored skills and 32 small fragments are waiting for domain review.

SECTION 3

Prompts used to build the ontology

Q18. What prompts did you use to generate the ontology?

All of them are in the **Prompts/** folder, in pipeline order. For the ontology: **01** turns the syllabus into the 183-topic taxonomy; **02** is the extraction system prompt (the rules); **03** is the task prompt (the per-batch procedure, a worked example, and a checklist); **04** is the editorial QA pass. **08** is the older prompt we replaced, kept for comparison.

Q19. Which model ran the ontology prompts?

Claude (Anthropic), run through Claude Code sessions that read the question files, extracted skills and ran our consolidation and validation scripts. Gemini was used only for question generation and verification.

If they push: be ready to say this plainly — the commit history shows Claude co-authorship on the ontology commits.

Q20. How was the extraction prompt designed? What are its key rules?

It uses role prompting (“You are OntologyGPT, an expert in HSC Mathematics, Physics and Chemistry at admission-test depth”) plus hard rules: one skill is one atomic, testable unit; one Bloom level per skill, set by the skill’s own verb; reuse an existing skill before creating a new one; file every skill under one of the 183 fixed topics; output a strict JSON format for skills and prerequisites.

Q21. Did the prompt stay the same for all 27 batches?

No. The biggest change came after the first Chemistry batches: only about 0.47 edges per skill, and 44% of skills had no prerequisite at all. The cause was the question we were asking: “what in this question depends on what else in this question?” — usually nothing, since a question yields only about one skill. We rewrote it to: “of everything extracted so far, what must a learner already know before this skill is reachable?” and added a gate: at least 1.0 edges per new skill, and a batch below 0.8 is not finished. Every batch after the change met the target.

Q22. Was there a worked example in the prompt?

Yes, prompt 03 walks through one real question, shows the skills extracted from it and the edges to earlier skills — “two skills, three edges; that ratio is normal” — so the model had a concrete picture of the granularity and edge density we wanted.

Q23. Why not just run the extraction in parallel to go faster?

We tried. The first attempt split the corpus into about 38 chunks with one sub-agent each, up to 16 at a time. It hit the account’s usage limit and failed after only 4 chunks. We switched to one long sequential session that kept the rules loaded and processed bigger batches — slower in wall-clock, but it finished, and the results were consistent because one session saw the whole growing ontology.

Q24. Did anything go wrong during extraction?

Yes, two things worth mentioning. A bug in our merge script reused skill IDs for topic codes containing digits, which created a fake cycle; the cycle check caught it and we replayed batches 11–14 from a snapshot. Earlier, LaTeX backslashes in the source text were being silently corrupted by JSON parsing and destroyed 334 records; we added a repair step and a rule to write formulas in words inside skill descriptions.

SECTION 4

Question bank generation and its prompts

Q25. How did you generate the question bank?

For each skill at each of the 6 Bloom levels — 10,128 specifications — we retrieve the 5 most similar real admission-test questions from a local vector database, and give them to Gemini as references with the skill, its Bloom level and its direct prerequisites. Gemini writes one new MCQ. A second model then checks it blind. Only questions that pass are loaded.

Q26. How many questions do you have?

About 12,300 verified questions, covering 94% (9,535) of the 10,128 skill × Bloom specifications; only 10 of 1,688 skills have no question at all. Coverage is lowest in Physics.

If they push: 10,128 is the number of specifications, not questions — don't mix them up.

Q27. Why generate every skill at all six Bloom levels?

Because the engine always asks one level above the learner's current band, so any skill can be asked at any level depending on the student. The downside, which we note ourselves: a pure recall fact gets a "Create"-level question, which can be artificial.

Q28. What is the retrieval step and why is it needed?

The corpus is embedded with a multilingual sentence model (paraphrase-multilingual-MiniLM-L12-v2, running locally) because the content is Bangla. We retrieve the top 5 matches, with a small boost for the same subject and for MCQs. This grounds the model in the real exam style and difficulty instead of generic textbook questions.

Q29. How was the generation prompt designed?

Role prompting ("You are an expert MCQ writer for Bangladesh university admission exams"); retrieval grounding; an explicit anti-copy rule ("Do NOT copy reference questions verbatim. Generate NEW questions for the requested skill"); a strict JSON schema, also enforced by Gemini's structured output; Bloom-level guidance; Bangla and LaTeX formatting rules; and MCQ-only, checked three times.

Q30. How are the distractors designed?

Each wrong option must come from one specific, nameable mistake a real candidate would make, and the prompt asks for each distractor to be built around missing one of the skill's direct prerequisites. The model tags the wrong option with that prerequisite, and we store the tag. When a student picks that option, the engine knows which prerequisite to suspect — that is what drives spillover.

If they push: not every wrong option has a tag: root skills have no prerequisites, and untagged distractors are allowed.

Q31. How do you verify that a generated question is correct?

A separate model call does two checks. First, is the question on-topic for the *exact* skill. Second, a blind re-solve: the verifier sees the stem and options but not the marked answer, solves it itself, and we compare. A question is kept only if it is on-topic and the verifier's answer doesn't disagree with the key.

Q32. Why a separate model for verification?

A model grading its own output tends to agree with itself. A second, independent call that never sees the answer gives real evidence. Folding the check into the generation call would have had the same bias.

Q33. What is the error rate of your questions?

We don't have a full pass-rate figure — the pipeline didn't log it. What we do have: a manual read of 36 questions found about 8% with a key problem the verifier can't catch (the keyed answer is only the closest option), and 8.3% of served questions have a rendering defect. Both are listed as open issues.

Q34. What did it cost?

Zero taka. It ran on Gemini free-tier keys, rotated with retry and back-off, and the embedding model runs locally on CPU.

If they push: we first tried local 7B/3B models on CPU, but the extrapolated run time was weeks, so we moved to Gemini.

The adaptive engine

Q35. What is Bayesian Knowledge Tracing, simply?

For every skill we keep one number: the probability the student has mastered it. After each answer we update it with Bayes' rule, accounting for two kinds of noise — a guess (right without knowing) and a slip (wrong despite knowing) — and then add a chance that they learned from the attempt.

Q36. Why BKT and not IRT or Deep Knowledge Tracing?

Four reasons. It gives an interpretable per-skill diagnosis, which is the whole product; IRT gives one ability number and DKT a hidden vector you can't read per skill. It uses our prerequisite graph explicitly. It works with **no historical data**, which we don't have before launch — IRT and DKT both need training data. And it is cheap to run. DKT is the natural successor once real learner data exists.

Q37. Why are guess and slip tied to Bloom level?

Higher-level questions are harder to guess and easier to slip on. So Remember uses guess 0.30 and slip 0.05, sliding to Create at guess 0.05 and slip 0.25. The values come from the Bloom level of the *question* being answered, not a fixed property of the skill.

Q38. Where do your BKT parameters come from? Did you fit them?

No, they are hand-set pedagogical priors, not fitted to data — we had no learner data to fit them on. Fitting them is the first thing to do once there are real users.

Q39. What is ancestor pull-up?

If a student answers a skill correctly, they almost certainly know its prerequisites too. So a correct answer raises every ancestor to at least the child's mastery. Wrong answers never push ancestors down, because a wrong answer doesn't tell you which prerequisite failed.

Q40. What is the conjunctive gate?

A skill only learns at the normal rate ($T = 0.10$) once *all* of its parents are at 0.95. If any parent is below, it learns at 0.01. The student can still practise it, but mastery barely moves until the missing prerequisite is fixed — so the system won't let a gap pass.

Q41. How does the diagnostic work?

One question per skill in the section, capped at 30 and at least 5. It picks the skill whose answer tells us the most about the rest of the graph, asks one Bloom level above the student's current band, and updates with a faster learning rate (0.20). Starting priors depend on depth: 70% for root skills down to 20% for the deepest.

Q42. What is prerequisite spillover?

During practice, if 3 of the last 5 wrong answers point to the same missing prerequisite (Rule A), or accuracy over the last 8 is below 40% and a suspected prerequisite is below 0.60 (Rule B), the engine inserts 2 questions on that prerequisite and then goes back to the topic.

Q43. Where does practice start?

At the topic skill with the highest mastery that is still below 95% — the zone of proximal development: something the student is close to, not something far beyond them.

SECTION 6

Evaluation: how do you know it works?

Q44. How did you test the system?

Two layers. An automated harness of 29 checks runs the full diagnostic → mastery update → practice → spillover flow against an in-memory database. Then a live end-to-end check against the production question bank: 51 sections, 4,862 requests, 14,066 assertions — 979 diagnostic answers all graded correctly, and spillover fired 84 times and served 130 prerequisite questions. 4 of 77 checks failed, and we report those as open defects.

Q45. Did you test it with real students?

No. There was no user study or classroom trial, so we can't claim a learning gain. What we can show is that the mechanisms behave as designed. A controlled study against a non-adaptive baseline is the main piece of future work.

Q46. Then how can you claim it helps students?

We claim less than that. We claim it correctly identifies and targets prerequisite gaps according to its model, and the design is grounded in established results — mastery learning, one-to-one tutoring, BKT. Whether that improves exam scores needs a study we haven't run.

Q47. How fast is it?

With one client: median 12 ms to submit an answer and 180 ms to pick the next question. The slow tail — about 2 s at the 95th percentile — comes from thin Physics sections where the fallback search has to look harder. We haven't load-tested multiple users.

Q48. What known defects did the evaluation find?

Be upfront: when a fallback question at a different level is served, it's still scored as if it were the target level; the parent-above-child ordering can be broken by later wrong answers in 4–8% of edges; some diagnostics end shorter than advertised; and 8.3% of questions have rendering issues. All are documented in the report.

SECTION 7

Admin and research features

Q49. What can an admin do?

Browse and edit the skill graph — one subject's topic map first, then drill into topics — and add or edit skills, topics and prerequisite edges; the editor refuses cycles and duplicate edges. They can also see class-wide mastery statistics and export mastery data as CSV or JSON for research.

Q50. If an admin edits the graph, do students see it immediately?

No, deliberately. Edits go to the ontology source files. They reach students only after the catalog is rebuilt and validated, so a half-finished edit can't break a live student's mastery state.

SECTION 8

Limitations and future work

Q51. What are the main limitations?

No real-student evaluation; BKT parameters are priors, not fitted; the ontology hasn't had domain-expert review; no forgetting model, so mastery never decays; and prototype shortcuts — username-only login and sessions held in memory.

Q52. What would you do next?

A classroom study against a non-adaptive baseline; fit guess, slip and learning rate from the logged answers (which requires logging every attempt, not just current mastery); a chemist and physicist review of the flagged parts of the graph; and later, move from BKT to Deep Knowledge Tracing once there is enough data.

Q53. Is the system secure enough to deploy?

Not yet. Login is username-only and admin access is a username allow-list; that was a deliberate prototype shortcut. Real authentication is required before any deployment with real students.

SECTION 9

Curveball questions

Q54. You used LLMs for both the ontology and the questions. Isn't that circular — AI checking AI?

Partly, and we designed against it. The ontology was extracted from real exam questions and checked by deterministic scripts, not by another LLM's opinion. For questions, the verifier is a separate call that never sees the answer. But yes, both still need human expert review, and we say so.

Q55. What exactly was your own contribution if an LLM did the extraction?

The LLM did the reading; we designed the process around it: the syllabus taxonomy, the extraction rules and quality gates, the consolidation and validation scripts, the decisions on every flagged case, and the diagnosis of why the first run came out sparse. And separately, the engine, the diagnostic, spillover, the question pipeline and the app.

Q56. Why Bangla content with English skill descriptions?

Students read and answer in Bangla, since that is the exam medium for most of them. Skill descriptions and prerequisite tags are internal and English so the graph is consistent and easy to review.

Q57. What happens if a student just guesses randomly?

Guess probability is built into BKT, so a lucky correct answer moves mastery less than a real one, especially on low-level questions where guessing is easiest. Over several questions, random answering keeps mastery low and the gate keeps downstream skills from advancing.

Q58. How many skills per topic, and is that granularity right?

Median about 7 per topic, much finer than the old ontology's 3. We targeted a size useful for diagnosis, but we haven't validated the grain with experts, and four Chemistry topics are above 40 skills. It's an open question we acknowledge.